AutoSaddler: Automatic Harness Optimization with Durable Updates from Agent Execution Traces

Sungho Park, Wonjoong Kim, Rongyuan Tan, Jue Zhang, Wook-Shin Han et al. · POSTECH · KAIST · SUSTech · Microsoft

HarnessRSI 79 ★★★

evidence: retained

Paper

Code

1 What changes, what is kept

Harness θ = prompt + tools + middleware, edited as code

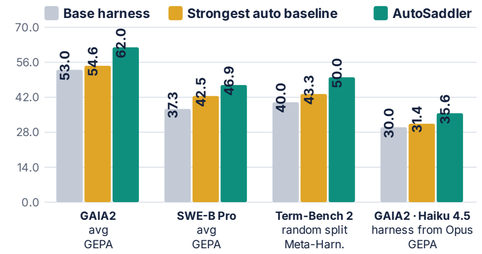
Prompt	Tool	Middleware
rules in system prompt text only (S)	new tool, args, fixes code (C) · docstring (S)	hooks, agent loop code (C) · hook text (S)

1. **Run a mini-batch**
   current harness on 6 training scenarios (GAIA2); traces collected
2. **Diagnose + patch**
   agent reads traces and harness source, writes a typed patch; capability patches first, then steering
3. **Verify, then gate**
   re-run the SAME batch; only an improvement goes on to the 65-scenario dev set
4. **Reflect**
   fixed / regressed / still-failing / still-passing → lessons into EvoDAG
5. **Evolve**
   agent composes the next harness from any earlier node: cherry-pick or roll back

50×

best dev score → one held-out test, 3 runs
weights never change; the kept artifact is the harness

2 Headline: held-out test Pass@1



Test sets: GAIA2 / SWE-B Pro = unseen universes / repos; **Terminal-Bench = random split, 40 tasks**. Mean of 3 test runs, one optimization run per method. Last group: Opus-optimized harness on a Haiku 4.5 agent. Dev gain is larger: GAIA2 dev 47.7‡ → 72.3 vs test 53.0 → 62.0; TB2 dev 52.6 → 73.7 vs test 40.0 → 50.0.

3 Controls that decide if the gain is real

Ablation · GAIA2 test Pass@1 (Table 2)



One re-optimization per variant; test sd 3.8–7.6. A 2nd AutoSaddler run alone moves 2.1 (Table 7): "w/o diagnosis" 57.1 is within that spread of 58.6 (U22), and 50.6 vs 53.0 is within sd 4.0.

Univ. 22	run 1	run 2
Default agent	51.5	
AutoSaddler	**60.7**	**58.6**
GEPA	47.9	50.6
Meta-Harness	51.5	51.2

Fine-grained	Pass@1
AutoSaddler	**60.7**
no phase sched.	54.8
no dev filter	50.0
+ no reflection	**44.9**

Reruns exist only for GAIA2 U22 (none for SWE-B Pro, TB2). Different train universe: 57.4.

Not controlled: baselines lack the gate

The dev-set gate + reflection removed in the last ablation row were not given to GEPA / Meta-Harness (no baseline-with-gate run). Budget: AutoSaddler and GEPA both 2 epochs; Meta-Harness set to ≥ as many task executions, and given train ∪ dev data. Fig 1a: AutoSaddler 72.3% dev at ~1,000 executions; GEPA ends ~1,600 (64.6%); only Meta-Harness runs to ~2,800 (61.5%), though the caption says both. Cost/patch: $14.56 vs $12.65 vs $5.50.

4 Where the gain is absent or thin

- **SWE-Bench Pro rests on one repo:** Ansible 58.0 vs base 40.6; Flipt 36.5 vs 31.0 (GEPA 32.2, Meta-Harness 31.3 ≈ base); Element-web 43.5 vs GEPA 45.2
- GAIA2 Universe 21: 61.4 vs GEPA 60.1 (sd 2.4 / 3.9). Meta-Harness ≈ default on GAIA2 (53.2 vs 53.0), below it on SWE-B Pro (35.3 vs 37.3)
- Terminal-Bench 2.0: random 30 / 19 / 40 split, sd 0.0; headline = Iter34 row (Iter2: 45.0), reason not stated
- **Text vs table (§5.2):** +8.4 / +6.2 / +4.4 printed; Table 3 gives 9.6 / 4.4 / 6.7 (SBP vs base, SBP vs best, TB2 vs best)

Takeaway

Patching prompts, tools and middleware from traces, verifying on the same batch and gating on a dev set, lifts three agent benchmarks by 9–10 points over base harnesses. The dev-set gate is the largest ablation effect, but baselines never got it, and evidence is one optimization run per method.

Our read · HarnessRSI

Score 79 · ★★★ · evidence level: retained

Reports: held-out test (TB2 random), 3 runs, ablations, cost.
Does not report: multi-run optimization, leakage test, baselines with the gate.